DiP: A Scalable, Energy-Efficient Systolic Array for Matrix Multiplication Acceleration

Ahmed J. Abdelmaksoud, *Member, IEEE*, Shady Agwa, *Member, IEEE*, and Themis Prodromakis, *Senior Member, IEEE*

***Abstract*—Transformers are gaining increasing attention across Natural Language Processing (NLP) application domains due to their outstanding accuracy. However, these data-intensive models add significant performance demands to the existing computing architectures. Systolic array architectures, adopted by commercial AI computing platforms like Google TPUs, offer energy-efficient data reuse but face throughput and energy penalties due to input-output synchronization via First-In-First-Out (FIFO) buffers. This paper proposes a novel scalable systolic array architecture featuring Diagonal-Input and Permutated weight stationary (DiP) dataflow for matrix multiplication acceleration. The proposed architecture eliminates the synchronization FIFOs required by state-of-the-art weight stationary systolic arrays. Beyond the area, power, and energy savings achieved by eliminating these FIFOs, DiP architecture maximizes the computational resource utilization, achieving up to 50% throughput improvement over conventional weight stationary architectures. Analytical models are developed for both weight stationary and DiP architectures, including latency, throughput, time to full PEs utilization (TFPU), and FIFOs overhead. A comprehensive hardware design space exploration using 22nm commercial technology demonstrates DiP's scalability advantages, achieving up to a 2.02× improvement in energy efficiency per area. Furthermore, DiP outperforms TPU-like architectures on transformer workloads from widely-used models, delivering energy improvement up to 1.81× and latency improvement up to 1.49×. At a 64×64 size with 4096 PEs, DiP achieves a peak throughput of 8.192 TOPS with energy efficiency 9.548 TOPS/W.**

***Index Terms*—Hardware acceleration, systolic arrays, spatial architectures, matrix multiplication, weight stationary.**

I. Introduction

ARTIFICIAL Intelligence (AI) is increasingly dominating various application domains [1]. Natural Language Processing (NLP) is one of the crucial AI application domains receiving increasing attention nowadays [2], [3]. Thanks to Transformers, NLP tasks have been revolutionized by providing highly effective and scalable models for language understanding and generation [4]. The exceptional capabilities of these models have led to a transformer-driven transformation across numerous application domains, including Machine Translation [5], Speech Recognition [6], Multimodal Applications [7], and Computer Vision [8].

However, transformers are data-intensive models that handle massive workloads compared to Deep Neural Networks (DNNs) and Convolutional Neural Networks (CNNs) [9]. Additionally, transformer models have been growing exponentially, evolving from the original vanilla Transformer model with around 65 million parameters to models with hundreds of billions of parameters [10], [11]. A clear example of the challenging level of scalability is GPT (Generative Pre-trained Transformer) model that incorporates billions of parameters, primarily involving matrix multiplications [12].

Conventional Von-Neumann architectures are struggling to meet these increasing performance demands due to the memory/data-movement bottleneck. Systolic arrays, introduced in 1982, are spatial architectures that designed to maximize the data utilization to mitigate the memory bottleneck. These architectures are receiving increased attention nowadays as a promising architecture for AI hardware acceleration [13]. Typically, a systolic array consists of a two-dimensional (2D) interconnected Processing Elements (PEs). The PE consists of basic arithmetic, mainly multiplication and accumulation, along with register units. Systolic arrays are spatial architectures that enhance local data utilization by increasing the number of computation operations per each memory access. Accordingly, input data circulates among PEs in a wave-like fashion, while the communication with the synchronization First-In-First-Out (FIFO)s occurs only at the boundary PEs. Moreover, the interconnection of the systolic array naturally facilitates data reusability among PEs, particularly during matrix multiplications [14].

Although many systolic arrays are used for hardware acceleration [15], [16], [17], [18], [19], [20], [21], different adopted dataflows require additional synchronization hardware, which limits the energy efficiency and constrains performance capabilities. Therefore, transformers with massive matrix multiplication workloads will face significant challenges in leveraging the systolic arrays for the next generation of AI hardware.

Tensor Processing Unit (TPU) is a well-known AI computing architectures introduced by Google to handle massive matrix multiplication workloads with higher performance and energy efficiency than CPUs and GPUs [22]. TPUs adopt weight stationary (WS) dataflow, which maximizes the data utilization of both weights and inputs. Google TPU V1 is designed primarily for inference, featuring a 256×256 systolic array optimized for 8-bit integer (INT8) operations,

Received 12 December 2024; revised 18 February 2025, 19 May 2025, and 14 July 2025; accepted 19 July 2025. Date of publication 28 July 2025; date of current version 12 January 2026. This work was supported in part by the Engineering and Physical Sciences Research Council (EPSRC) Programme Grant "Functional Oxide Reconfigurable Technologies" (FORTE) Program under Grant EP/R024642/2 and in part by the RAEng Chair in Emerging Technologies under Grant CiET1819/2/93. This article was recommended by Associate Editor H. Cai. *(Corresponding author: Ahmed J. Abdelmaksoud.)*

The authors are with the Centre for Electronics Frontiers, Institute for Integrated Micro and Nano Systems, School of Engineering, The University of Edinburgh, EH9 3BF Edinburgh, U.K. (e-mail: a.j.abdelmaksoud@ed.ac.uk; shady.agwa@ed.ac.uk; t.prodromakis@ed.ac.uk).

Digital Object Identifier 10.1109/TCSI.2025.3591960

achieving a peak throughput of 92 TOPS [23]. TPU V2 shifted to mixed-precision training with a smaller 128 × 128 systolic array per core optimized for FP16 and bfloat16 operations, boosting throughput to 180 TeraFLOPS [24]. TPU V3 and V4 maintained the same array size but TPU V3 doubled the throughput to 420 TFLOPS per chip, aided by high memory bandwidth, while TPU v4 integrates four cores of 128 × 128 architecture, achieving up to 1 PFLOPS [25].

Meissa is a systolic architecture that separates multipliers from adders instead of integrating them into a unified array [26]. Similar to TPUs, it employs WS dataflow for the systolic array but eliminates input synchronization FIFOs to reduce overall latency. However, it has bulky adder trees per each column of the systolic array. These adder trees impose scalability limitations due to serious physical implementation challenges. As adder trees become larger, they require deeper pipelines to achieve higher frequencies. This increases the overall latency, area, and energy consumption. In addition, routing congestion presents another costly challenge, when all partial products from each PE column are delivered to the adder tree. Consequently, Meissa is not scalable to large N × N dimensions, which is critical for computation-intensive workload. Moreover, it still requires the output synchronization FIFOs, which add additional area, power, and energy penalty.

In this paper, we present a novel Diagonal-Input Permutated (DiP) weight stationary systolic array that overcomes the main challenges of the conventional WS systolic array architectures. The main contributions of this work are highlighted as follows:

- We introduce DiP, a novel scalable spatial architecture of N × N PEs for matrix multiplication acceleration. DiP maximizes PE utilization and energy efficiency, achieving a throughput improvement of up to 1.49×.
- The proposed architecture eliminates the input/output synchronization FIFOs in WS by implementing a new dataflow with diagonal-input movement and permutated weights.
- The analytical models are extracted for DiP and WS including throughput, latency, time to full PE utilization (TFPU), and register overhead for different systolic array sizes.
- Hardware design space exploration and implementation are presented for DiP and WS using commercial 22nm technology, offering up to 2.02× energy efficiency per area improvement.
- DiP is evaluated using various transformer workloads from widely used models, outperforming TPU-like architectures and achieving up to 1.81× energy efficiency improvement and up to 1.49× latency improvement.

This paper is organized as follows; Section II discusses the systolic arrays background. Section III presents DiP architecture. Section IV shows hardware design space exploration, evaluation, and results. Finally, section V concludes the work.

II. Systolic Arrays Background

Systolic arrays adopt one of the dataflows to control the data movement across the PEs. Each dataflow retains one of the input, output, or weight data to be stationary during computations for the maximum duration to maximize the data reuse [27], [28]. The following dataflows are the most common in systolic array design:

- Weight Stationary (WS): weights are initially loaded to the PEs and kept stationary during processing. The input matrix and partial summations move among the PEs.
- Input Stationary (IS): input matrix is initially loaded to the systolic array, while weight matrix and partial summations move among the PEs.
- Output Stationary (OS): input and weight matrices move among the PEs, while the partial summations are accumulated inside PEs.
- Row Stationary (RS): it is proposed by Eyeriss [29]. It adopts spatial architecture that uses coarse-grained PEs with internal memories to store weights and inputs. Inputs are broadcasted diagonally across the PEs, while weight matrix broadcasted horizontally, and partial summations move vertically.

The OS dataflow moves both input and weight matrices simultaneously, which effectively doubles the required memory bandwidth for the systolic array. In RS dataflow, data redundancy increases because copies of the data are loaded into multiple PEs. Additionally, the circulation of weights within each PE reduces energy efficiency. On the other hand, WS dataflow is widely used in many architectures, such as Google TPU, due to its scalability and flexibility in handling convolutions and matrix multiplication [23], [24], [25]. Moreover, it requires less memory bandwidth. Therefore, we focus on improving this dataflow.

A. WS Dataflow

The WS dataflow is widely used in many architectures where weights are initially loaded to the PEs, while the input matrix circulates between them in a systolic fashion. This approach mitigates the memory bottleneck by reducing memory accesses and increasing data reuse. However, WS dataflow requires input and output FIFOs to synchronize data for proper functionality. Figure 1 shows the architecture of WS systolic array consisting of N × N PEs and two FIFO groups for input/output synchronization. WS PE consists of 2-stage pipelined MAC unit and four enabled registers for input, weight, multiplier output and adder output. Control signals *wshift*, *pe_en*, *mul_en*, and *adder_en* manage the PE operations. Specifically, *wshift* enables the weight register, while *pe_en* enables the input register. *mul_en* and *adder_en* selectively enable their respective registers only during active computation cycles, reducing power consumption during inactive cycles. *wshift* is shared across each PE row, while *pe_en*, *mul_en*, and *adder_en* are shared across each PE column. The input FIFO group consists of series of FIFOs with incrementally increasing depth starting from one element in the second row to $N-1$ elements in the last row. The output FIFO group is structured similarly to the input FIFO group, but the FIFO depths decrease from $N-1$ elements to one element moving from left to right across the systolic array columns.

The input FIFOs map the input matrix to the PEs in diagonal wave fashion, while the output FIFOs synchronize

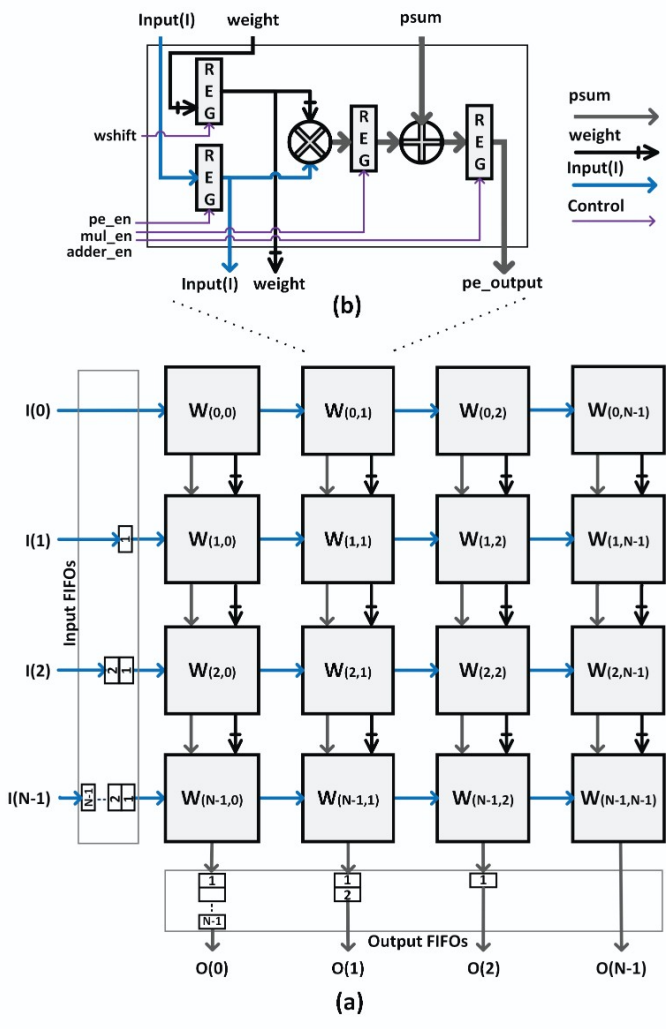

Fig. 1. (a) Top-level schematic for the WS systolic array with N × N PEs and two input/output FIFO groups. The weights (black crossed buses) are loaded vertically, and psums (grey buses) are accumulated along the columns. (b) WS PE block diagram, consisting of a 2-stage pipelined MAC unit and four enabled registers. Control signals *wshift*, *pe_en*, *mul_en*, and *adder_en* are used for operations control. *wshift* is shared across each PE row, while *pe_en*, *mul_en*, and *adder_en* are shared across each PE column.

partial summations along each column [27]. In WS, the input matrix and partial summations (psums) are pipelined to ensure proper processing and improve data reuse. The PE in WS is fine-grained, unlike the coarse-grained PE in Eyeriss, which broadcasts weights and input matrices to the array and stores them in internal memories [29]. Weights are loaded vertically, while the input matrix is shifted horizontally, while the output is shifted to the output FIFO group. Using synchronization FIFOs add significant overheads in terms of area, power, energy consumption, and latency during matrix multiplication. Moreover, the computations propagate as a diagonal wave from the top-left corner to the bottom-right corner of the systolic array. This substantially reduces overall PE utilization resulting in degraded performance and increased latency. As a result, FIFO-based systolic arrays suffer from lower energy efficiency, reduced PE utilization, higher latency, and decreased throughput.

The analytical modeling of the WS systolic array is studied for latency, throughput, TFPU, and FIFOs overhead, providing insights into dataflow performance. For the WS latency analytical model, as shown in (1), WS requires $3N+S-3$ cycles to complete the processing, where N represents the number of rows/columns per WS systolic array, and S is the number of pipeline stages per Multiply-Accumulate (MAC) unit, which equals one for 1-stage pipelined MAC, and two for 2-stage pipelined MAC. The throughput, as indicated in (2), is calculated as the ratio of total operations to WS latency. Regarding register overhead, WS systolic array uses two FIFO groups for input and output synchronization. Each group consists of $N-1$ FIFOs, as shown in Fig. 1. Additionally, each FIFO group includes $N(N-1)/2$ registers. Consequently, the total register overhead for a typical WS systolic array is calculated as shown in (3). TFPU is another metric introduced to calculate the required number of cycles to reach full utilization of PEs. This metric shows the overhead when the input matrix is initially loaded to the PEs. TFPU is calculated, as shown in (4), where it takes $2N-1$ cycles for WS to reach full PEs utilization.

$$\text{Latency for WS} = 3N + S - 3 \tag{1}$$

$$\text{Throughput for WS} = \frac{2N^3}{3N+S-3} \tag{2}$$

$$\text{Registers overhead for WS} = N(N-1) \tag{3}$$

$$\text{TFPU for WS} = 2N - 1 \tag{4}$$

III. DiP Architecture

In this section, we discuss DiP architecture, DiP dataflow, and DiP analytical models. Then, we compare the analytical models for DiP and WS.

A. DiP Architecture

DiP is a scalable spatial architecture consisting of N × N PEs, designed to accelerate matrix multiplication computations, as shown in Fig. 2(a). The input matrix moves diagonally across the PEs, passing from one PE row to the next. The boundary PEs are diagonally connected, with the registered inputs of the leftmost PE column are connected to the inputs of the rightmost PE column in the subsequent row. The weight matrix is loaded vertically, and psums are accumulated vertically. Figure 2(b) shows the architecture of each PE.

The proposed PE is similar to WS PE, utilizing a 2-stage pipelined MAC unit and four enabled registers for input, weight, multiplier output and adder output. Control signals *wshift*, *pe_en*, *mul_en*, and *adder_en* manage the PE operations. Specifically, *wshift* enables the weight register, while *pe_en* enables the input register. *mul_en* and *adder_en* selectively enable their respective registers only during active computation cycles, reducing power consumption during inactive cycles. However in DiP, *wshift*, *pe_en*, *mul_en*, and *adder_en* are shared across each PE row.

B. DiP Dataflow

The proposed DiP architecture adopts a novel dataflow to control the movement of inputs, weights, and partial summations across the whole systolic array. DiP dataflow relies on

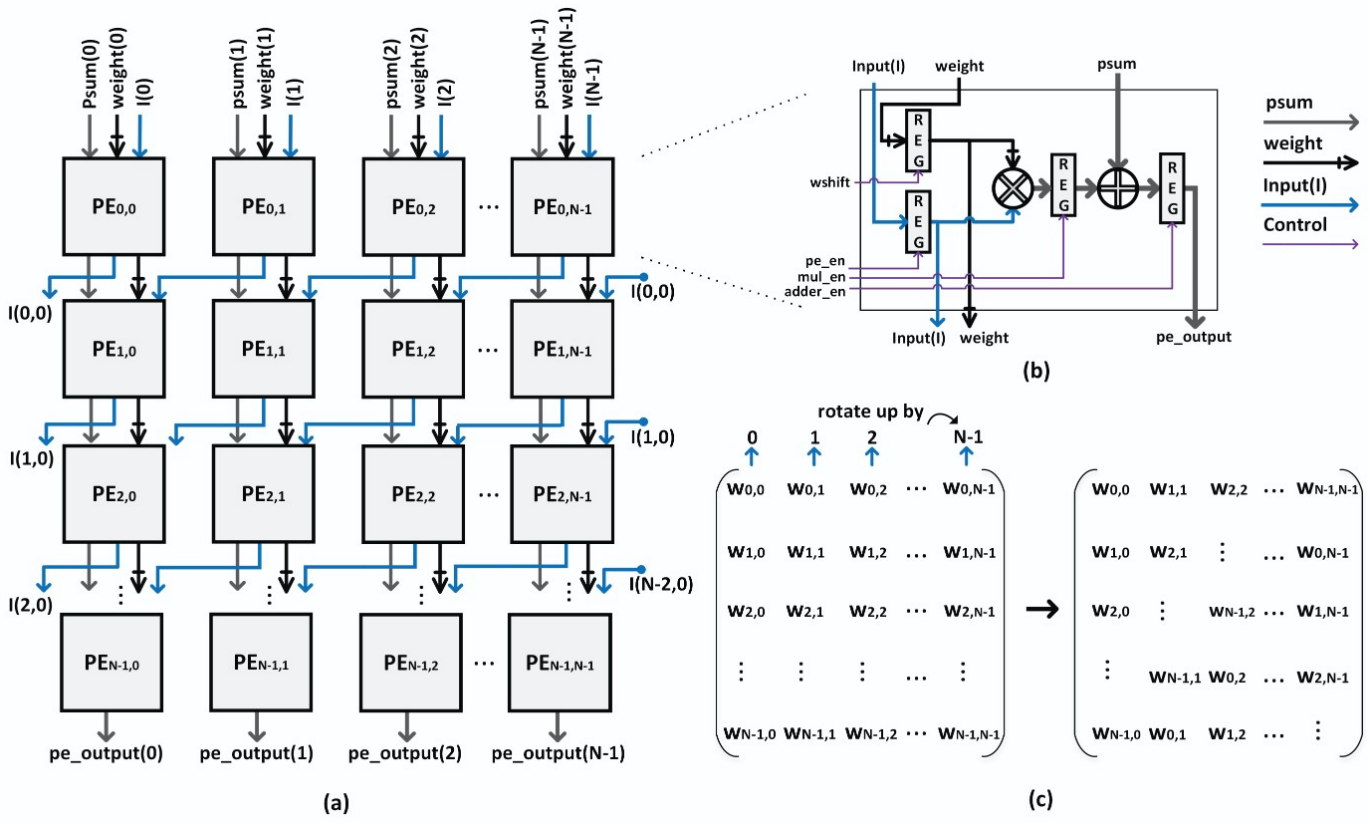

Fig. 2. (a) N × N DiP systolic array architecture, inputs(I) move diagonally across PE rows, transitioning from one row to the next. The boundary PEs are diagonally connected, so that the registered inputs from the leftmost PE column feed into the inputs of the rightmost PE column in the subsequent row. Weights are loaded vertically, and psums are accumulated vertically along the columns as well. (b) PE block diagram, consisting of a 2-stage pipelined MAC unit and four enabled registers. Control signals *wshift*, *pe_en*, *mul_en*, and *adder_en* are used for operations control. *wshift* is shared between all systolic array PEs, while *pe_en*, *mul_en*, and *adder_en* are shared across each PE row. (c) General weight matrix permutation for DiP dataflow. The weight matrix is permutated by shifting and rotating each column by its column index.

Algorithm 1 Matrix Permutation for DiP Dataflow

Input: *matrix*
Output: *permutated_matrix*
  for $i \leftarrow 0$ to cols **do**
    for $j \leftarrow 0$ to rows **do**
      $permutated\_matrix[j][i] = matrix[(j+i)\%rows][i]$
    end for
  end for

two major upgrades compared to WS dataflow, the diagonal movement of input matrix, and the weight matrix permutation. Firstly, the input matrix moves diagonally across PE rows, as shown in Fig. 2(a). Secondly, the proposed dataflow permutates the weights by shifting and rotating each column by its column index, as shown in Fig. 2(c). The weights permutation is initially prepared on software level, as shown in Algorithm 1. For each column, it iterates over all rows, assigning each element in the *permutated_matrix* based on its row and column index. The permutation is performed at the software level when the second matrix is a weight matrix. Alternatively, it is executed at run-time by efficiently re-scheduling memory access across multi-bank memories with almost zero overhead. The proposed dataflow removes the need for input and output synchronization FIFOs typically required by conventional WS systolic arrays. Additionally, it enhances throughput and PE utilization while reducing chip area and latency.

Figure 3 shows a complete example for 3 × 3 DiP systolic array. It consists of three rows: a) Row-0: PE_{00}, PE_{01}, PE_{02}, b) Row-1: PE_{10}, PE_{11}, PE_{12}, c) Row-2: PE_{20}, PE_{21}, PE_{22}. The PE array is diagonally connected, as shown in Fig. 3(a). The leftmost PE in each PE row is connected to the rightmost PE in the next PE row. The weight matrix is initially permutated to be prepared for weights loading. Each column is shifted and rotated by its column index, as shown in Fig. 3(b). The weights are initially loaded, row by row, to the systolic array, as shown in Fig. 3 from *Cycle -2* to *Cycle 0*. The loading of the last weight row and the first input matrix are performed in parallel at *Cycle 0*. The input data is loaded in parallel to Row-0 including inputs for PE_{00}, PE_{01}, and PE_{02}. After accomplishing the required computations by Row-0, the input data is shifted diagonally from PE_{00} to PE_{12} and from PE_{01} to PE_{10} and from PE_{02} to PE_{11}. Then after accomplishing the required computations by Row-1, the input data is shifted diagonally again from PE_{10} to PE_{22} and from PE_{11} to PE_{20} and from PE_{12} to PE_{21}. The processing starts from *Cycle 1* to *Cycle 5*, while the first output row becomes ready at *Cycle 3*, and the last output row becomes ready at *Cycle 5*. The processing goes as follows:

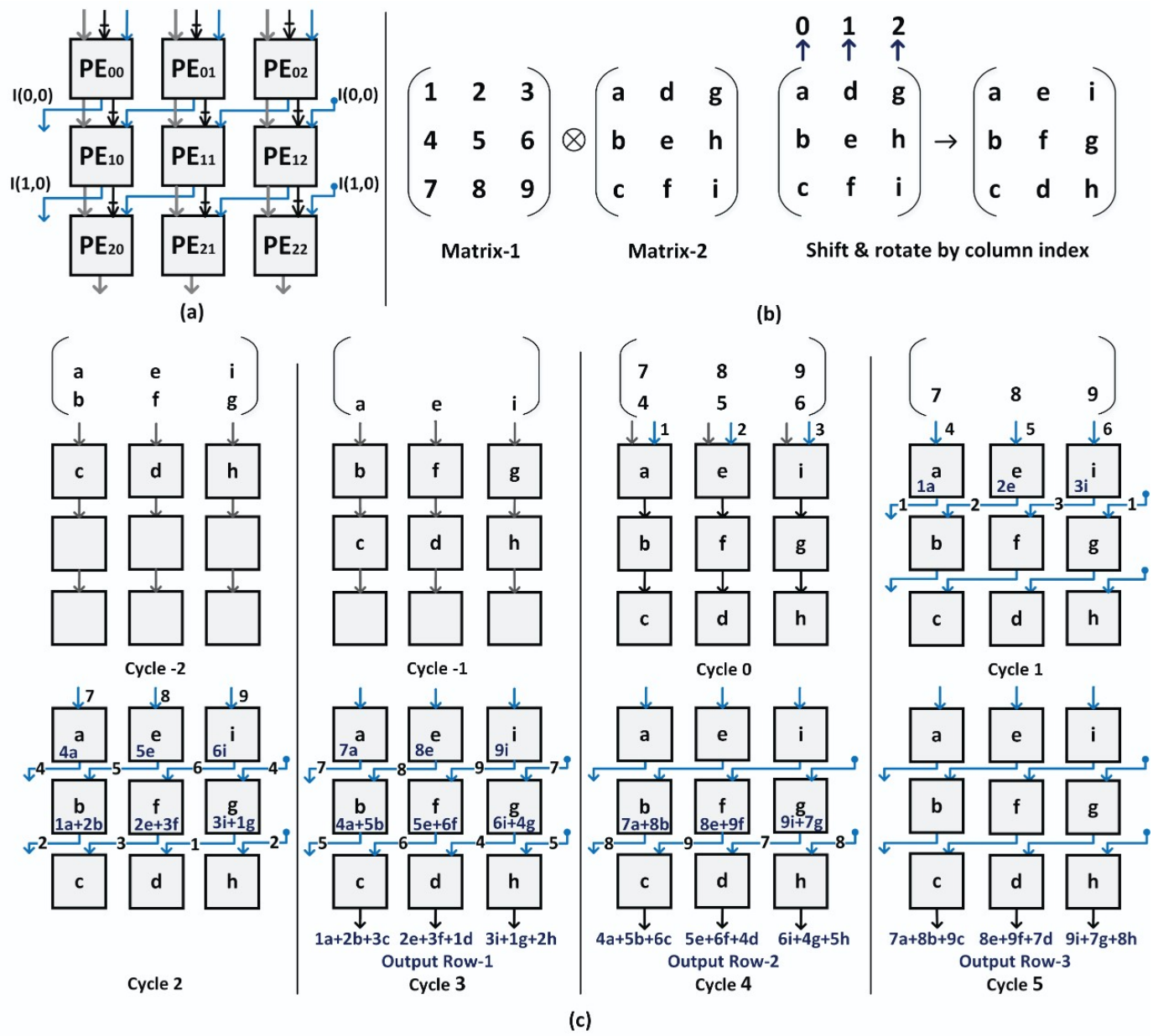

Fig. 3. Example for 3 × 3 DiP systolic array (a) shows the diagonal input connections for 3 × 3 DiP, (b) shows 3 × 3 DiP weight matrix permutation by shifting and rotating each column by its column index, and (c) shows the processing flow for 3 × 3 DiP example cycle by cycle. *Cycles (-2, -1, 0)* are dedicated to weight matrix loading, while *Cycle 0* involves loading the last row of the weight matrix and the first row of the input matrix. Cycles from *Cycle 1* to *Cycle 5* are allocated for matrix multiplication processing, while the first output row becomes ready at *Cycle 3*.

- **Cycle -2**: The last row of weight matrix *(c, d, h)* is loaded to the first PE row.
- **Cycle -1**: The weights matrix row *(c, d, h)* is shifted to the second PE row, and new weights row *(b, f, g)* is loaded to the first PE row.
- **Cycle 0**: The weight matrix row *(c, d, h)* is shifted to the last PE row, the weights in the first PE row *(b, f, g)* are shifted to the second PE row, and new weights row *(a, e, i)* and the first input matrix row *(1, 2, 3)* are loaded to the first PE row, simultaneously.
- **Cycle 1**: The first PE row shifts the partial summations *(1a, 2e, 3i)* to the second row. Using the diagonal connections, the input matrix row *(1, 2, 3)* is permutated to *(2, 3, 1)*, and loaded to the second PE row at the same cycle.
- **Cycle 2**: The first PE row shifts the partial summations *(4a, 5e, 6i)* to the second row, and the input matrix row *(4, 5, 6)* is permutated to *(5, 6, 4)* and loaded to the second PE row. Similarly, The second PE row shift the partial summations *(1a+2b, 2e+3f, 3i+1g)* to the third row, and the input matrix row *(2, 3, 1)* is permutated to *(3, 1, 2)* and loaded to the third PE row.
- **Cycle 3**: The first PE row shift the partial summations *(7a, 8e, 9i)* to the second row, and the input matrix row *(7, 8, 9)* is permutated to *(8, 9, 7)* and loaded to the second PE row. Similarly, The second PE row shift the